

■ Guía Paso a Paso: Descifrando URDF, `robot\_description` y TF2 en Burger Delivery

¡Bienvenido! En esta guía vamos a explorar juntos el corazón de la descripción robótica de nuestro proyecto. El objetivo es que aprendas a leer, validar y ajustar la estructura de `burger\_delivery` sin tratar el URDF como un simple "XML decorativo".

■ Contexto: ¿Qué es realmente un URDF?

Antes de tocar el código, es vital entender qué estamos haciendo. **URDF** (*Unified Robot Description Format*) es, literalmente, el **ADN de tu robot**.

En robótica, no basta con tener un modelo 3D bonito. El ordenador necesita un **modelo matemático** para responder preguntas críticas:

- ¿Hacia dónde puede girar este brazo?
- ¿Si muevo el motor 10 grados, dónde quedará la pinza?
- ¿Esta pieza chocará con la mesa si avanzo?

El URDF traduce la realidad física a un lenguaje que ROS 2 entiende, dividiendo al robot en dos elementos básicos:

1. **Links (Eslabones):** Son las partes rígidas (los "huesos"). Tienen peso, apariencia y volumen.
2. **Joints (Articulaciones):** Son las uniones que permiten el movimiento (las "bisagras"). Definen cómo se conecta un link con otro.

Esta descripción es un **estándar en la industria**. Lo mejor de usar URDF es la **portabilidad**: el mismo archivo que usas hoy para ver el robot en RViz, lo usará MoveIt 2 para calcular rutas sin choques, y Gazebo o Ignition para simular la gravedad y los motores.

Sin un URDF bien configurado, el robot es solo un montón de metal ciego. Con él, se convierte en una entidad capaz de interactuar con su entorno con precisión milimétrica.

Cada sección de esta guía combina tres elementos clave:

1. **El Concepto:** ¿Qué estamos tratando de entender?
2. **La Realidad:** Revisión real en los archivos del proyecto.
3. **La Acción:** Ejercicios prácticos y verificables en terminal o RViz.

Nos enfocaremos en:

- La escena fija (`delivery\_scene\_fixed.urdf`).
- El manipulador Kinova Gen3.
- Los carritos modelados como URDFs separados.
- La conexión entre URDF, `robot\_description`, `robot\_state\_publisher` y TF2.
- La localización por AprilTags usando `tag\_mesa -> tag\_carrito`.

Un consejo antes de empezar: Tómate tu tiempo. No conviene avanzar de sección si no se cumple su criterio de éxito. Cada paso confirma un concepto que usaremos más adelante. ¡Vamos allá!

0. ¡Manos a la obra! Preparación y Primera Prueba

■ El Concepto

ROS 2 no carga archivos nuevos automáticamente desde `src`. Los archivos que usa `ros2 launch` se resuelven desde el paquete instalado en `install/share`. Si agregas, renombas o editas URDFs, es **obligatorio**

reconstruir el paquete.

■ Revisión en el Proyecto

El launch principal está en: ``burger_description/launch/display.launch.py``.

Este archivo es el encargado de cargar:

- ``burger_description/urdf/delivery_scene_fixed.urdf`` (Escena y Kinova).
- ``burger_description/urdf/car1_apriltag.urdf`` (Carrito 1).
- ``burger_description/urdf/car2_apriltag.urdf`` (Carrito 2).

■ Tu primer reto: Construir y Validar

Vamos a preparar tu entorno. Ejecuta estos comandos en tu terminal:

1. **Crea el enlace para los recursos (Meshes):**

Por un estándar de ROS, el URDF busca los modelos en una carpeta llamada ``meshes``. En nuestro proyecto, estos archivos están dentro de ``visual/meshes``. Ejecuta esto para crear el puente:

```
cd ~/ros2_ws/src/burger_delivery/burger_description
ln -s visual/meshes meshes
```

2. **Compila y refresca tu espacio de trabajo:**

```
cd ~/ros2_ws
colcon build --packages-select burger_description --symlink-install
source install/setup.bash
```

3. **Verifica la instalación real:**

```
ls -l install/burger_description/share/burger_description/urdf
ls -l install/burger_description/share/burger_description/meshes
```

Deben aparecer los archivos ``*.urdf`` y los archivos ``*.stl`` de la mesa.

4. **¡Lanza la visualización!**

Usaremos ``use_static_carts:=true`` para crear TFs temporales y ver los carritos:

```
ros2 launch burger_description display.launch.py use_static_carts:=true
```

■ ¿Cómo sé si voy por buen camino? (Criterio de Éxito)

- El launch no falla con ``FileNotFoundError``.
- RViz abre correctamente y ****puedes ver la mesa marrón y el brazo robot****.
- Puedes usar ``map`` como ``Fixed Frame``.
- Los TF temporales ``tag_mesa -> tag_carrito1`` y ``tag_mesa -> tag_carrito2`` son visibles en RViz.

1. El viaje del URDF hacia ``robot_description``

■ El Concepto

Un URDF describe la estructura como texto XML, pero los nodos no trabajan directamente con el archivo en disco. El flujo real es:

1. El **Launch** lee el archivo ``*.urdf`` como texto.
2. Ese texto se pasa como parámetro **`robot\_description`**.
3. El nodo **`robot\_state\_publisher`** lee ese parámetro y lo combina con ``/joint_states``.
4. Finalmente, ROS publica las transformaciones en ``/tf`` y ``/tf_static``.

■ Exploremos el código

Abre el launch para ver este flujo: ``code burger_description/launch/display.launch.py``.

Busca:

- Lectura de `delivery\_scene\_fixed.urdf` y de los carritos.
- Cada nodo `robot\_state\_publisher` y su parámetro `robot\_description`.

Comando rápido para encontrar estas líneas:

```
rg -n  
"robot_description|robot_state_publisher|car1_apriltag|car2_apriltag|delivery_scene_fixed"  
burger_description/launch/display.launch.py
```

■ Ejercicio: Inspección de Nodos

1. Lanza el sistema: `ros2 launch burger\_description display.launch.py use\_static\_carts:=true`
2. Verifica publicadores separados:

```
ros2 node list
```

(Deberías ver `robot\_state\_publisher`, `car1\_state\_publisher` y `car2\_state\_publisher`).

3. Comprueba el parámetro del publicador principal:

```
ros2 param get /robot_state_publisher robot_description | head -n 20
```

■ Criterios de Éxito

- Puedes explicar por qué hay más de un `robot\_state\_publisher`.
- Entiendes que si agregas un URDF nuevo al proyecto, debes actualizar el launch y reconstruir el paquete.

■ Nota Especial: El tramo temporal `use\_static\_carts:=true`

Es fundamental que entiendas que este modo es solo para depuración inicial.

- **¿Qué pregunta responde?** ¿El URDF del carrito se carga y se ve bien si recibe una pose?
- **¿Qué NO responde?** ¿La cámara detecta el tag? ¿La localización está calibrada?

En operación real, el responsable de ubicar los carritos no es el launch ni el URDF. Es el nodo de localización AprilTag, que publica en tiempo real: `tag\_mesa -> tag\_carrito`.

Modo	Quién publica `tag_mesa -> tag_carrito`	Resultado
Visualización (`true`)	El launch (estático)	Carritos aparecen fijos cerca de la mesa.
Esperando localización	Nadie	Carritos cargados, pero "perdidos" (no conectados al `map`).
Operación real	Nodo AprilTag	Carritos aparecen en su pose estimada real.
Simulación (Gazebo)	Plugin de Odometría	Carritos se mueven según las leyes de la física.

Regla de oro: No uses `use\_static\_carts:=true` al mismo tiempo que el nodo real de localización, o tendrás dos fuentes intentando publicar lo mismo, creando un árbol TF ambiguo.

> [!NOTE]

> Fundamento de Robótica: ¿Por qué tantos nodos publicando?

> En un sistema distribuido como ROS 2, no hay una "CPU central" que calcule todo. Cada `robot\_state\_publisher` es responsable de una pieza del rompecabezas. Esto permite que el sistema sea escalable: puedes agregar 10 carritos más simplemente lanzando 10 pequeños nodos publicadores, sin sobrecargar un solo proceso gigante.

** No modifiques el URDF para "pegar" el carrito a la mesa. Deja que el sistema de localización haga su trabajo.

2. Anatomía de un `Link`: Visual, Collision e Inertial

■ El Concepto

Un `link` representa una parte rígida. Conviene leerlo como tres capas distintas:

Bloque	Para qué sirve
`visual`	Geometría visible en RViz o simulador.
`collision`	Lo que usa MoveIt 2 o el motor de física para detectar choques.
`inertial`	Masa e inercia para la simulación dinámica.

Importante: El bloque `visual` no sirve para colisiones. En entornos profesionales, las colisiones suelen ser geometrías simples (cajas, cilindros) para que la planificación sea robusta y rápida.

> [!TIP]

> **Fundamento de Robótica: La importancia de la Inercia**

> Aunque para visualizar el robot no necesitamos el bloque `inertial`, para la **dinámica** es vital. Si quieres que el brazo Kinova se mueva de forma fluida en un simulador (o que el controlador real compense la gravedad), el sistema debe saber cuánto pesa cada link y cómo está distribuida su masa (momento de inercia). Sin esto, el brazo caería "como un peso muerto" o se movería de forma errática.

■ Revisión en el Proyecto

Busca estos links en `delivery\_scene\_fixed.urdf` usando:

```
rg -n '<link name="table_link"|<link name="staging_area"|<link name="delivery_slot_1"|<visual|<collision|<inertial' burger_description/urdf/delivery_scene_fixed.urdf
```

Link	Qué representa	`visual`	`collision`	Observación
`table_link`	Mesa física	Malla STL y ejes	Caja simple	La colisión no incluye los ejes visuales.
`staging_area`	Zona de preparación	Caja plana	Caja simple	Es geometría de escena simplificada.
`delivery_slot_1`	Zona objetivo	Caja visual	Caja simple	Sirve como volumen lógico.
`gen3_..._link`	Brazo Kinova	Malla CAD	Malla CAD	El robot usa geometría detallada.

■ Ejercicio en RViz

- Lanza: `ros2 launch burger_description display.launch.py use_static_carts:=true``
- En RViz, activa `RobotModel` y cambia entre vista de escena y TF.
- Identifica la mesa y el brazo. Activa `TF` y observa que los ejes visuales de `table\_link` son solo marcas de referencia, no sólidos de colisión.

■ Criterios de Éxito

- Distingues entre geometría de escena simplificada y geometría CAD del robot.
- Puedes explicar por qué `visual` y `collision` no deben copiarse ciegamente (ej: no usar mallas complejas para colisiones si basta con una caja).

■ Mini-Reto de Validación: ¿Visual o Colisión?

1. Abre ``burger_description/urdf/delivery_scene_fixed.urdf``.
2. Busca el link ``table_link``.
3. Cuenta: ¿Cuántos bloques ``<visual>`` tiene? ¿Y cuántos bloques ``<collision>``?
4. En RViz: Activa la visualización de colisiones (Panel ``Displays`` -> ``RobotModel`` -> ``Collision Enabled``).
5. Pregunta: ¿Ves las flechas de los ejes (roja, verde, azul) cuando solo activas las colisiones? ¿Por qué crees que es así?

3. ``Joint``: Tipos, Ejes, Límites y ``mimic``

■ El Concepto

Un ``joint`` conecta exactamente un ``parent link`` con un ``child link``. Define cómo puede moverse el hijo respecto al padre.

Tipos principales:

Tipo	Movimiento
<code>`fixed`</code>	Sin movimiento.
<code>`revolute`</code>	Rotación con límites angulares.
<code>`continuous`</code>	Rotación sin límite (como una rueda).
<code>`prismatic`</code>	Traslación lineal (deslizamiento).

> [!NOTE]

> Fundamento de Robótica: Grados de Libertad (DOF)

> Cada joint que no sea ``fixed`` añade un Grado de Libertad al robot. Nuestro Kinova Gen3 tiene 7 joints rotacionales, lo que significa que tiene 7-DOF. En robótica, tener 6-DOF es el mínimo para alcanzar cualquier posición (XYZ) con cualquier orientación (RPY) en el espacio. ¡Tener 7-DOF nos da "redundancia", permitiendo al robot evitar obstáculos sin mover su pinza del objetivo!

■ Revisión en el Proyecto

Busca los joints del Kinova en ``delivery_scene_fixed.urdf``:

```
rg -n 'gen3_joint_[1-7]|mimic|limit|axis' burger_description/urdf/delivery_scene_fixed.urdf
```

Verifica en el archivo:

- ``gen3_joint_2`` es ``revolute``, usa ``axis="0 0 1"`` y tiene límites ``lower="-2.41"`` y ``upper="2.41"``.
- ``gen3_joint_1`` (y otros) son ``continuous``.
- La pinza usa ``mimic`` con ``multiplier="-1"`` para que el dedo derecho imite al izquierdo en sentido opuesto.

■■ Ejercicio: ¡Mueve el robot!

1. Lanza: ``ros2 launch burger_description display.launch.py use_static_carts:=true``
2. En el panel Joint State Publisher, mueve los deslizadores.
3. Verifica en RViz: Mueve ``gen3_joint_2``. ¿Ves cómo el slider queda limitado aproximadamente en el rango del XML?
4. Prueba la pinza: Mueve ``gen3_robotiq_85_left_knuckle_joint``. Observa cómo el dedo derecho acompaña el movimiento gracias al ``mimic``.

■ Criterios de Éxito

- Ubicas un ``axis`` en el XML y lo relacionas con el movimiento en RViz.
- Explicas qué problema resuelve el uso de ``mimic`` en un mecanismo acoplado como una pinza.

■ ■ *Mini-Reto de Validación: Cazador de Límites*

1. Abre el archivo ``delivery_scene_fixed.urdf`` y busca el joint ``gen3_joint_2``.
2. **Investiga:** ¿Cuál es su valor de ``upper`` y ``lower`` limit?
3. Ahora busca ``gen3_joint_1``. ¿Tiene límites numéricos? ¿Por qué?
4. **En la terminal:** Mientras el sistema corre, ejecuta:

```
ros2 topic echo /joint_states
```

5. Mueve el robot en el GUI y observa cómo cambian los números en la terminal. ¿Los nombres de los joints coinciden con lo que viste en el XML?

4. Árbol Cinemático: map, table_link, world y Kinova

■ *El Concepto*

URDF describe un árbol. Cada ``link``, salvo la raíz (``map``), debe tener un solo padre. TF2 compone estas transformaciones para saber dónde está cualquier pieza respecto a otra.

> [!IMPORTANT]

> **Fundamento de Robótica: Cinemática Directa (Forward Kinematics)**

> El árbol cinemático es la base para calcular la **Cinemática Directa**. Es el proceso matemático que responde a: *"Si conozco los ángulos de todos los joints, ¿dónde está exactamente la punta del robot?"*. TF2 hace este cálculo por nosotros millones de veces por segundo sumando las transformaciones desde la base hasta la punta.

■ *Revisión en el Proyecto*

Busca las conexiones clave en ``delivery_scene_fixed.urdf``:

```
rg -n 'map_to_table|table_to_world|table_to_kinova_base|tool_to_grip|table_to_tag_mesa'
burger_description/urdf/delivery_scene_fixed.urdf
```

Cadena clave	Propósito
<code>`map -> table_link`</code>	Ubicación de la mesa en el mundo global.
<code>`table_link -> world -> gen3_base_link`</code>	Inserción del brazo Kinova en la mesa.
<code>`kinova_tool_frame -> burger_grip_frame`</code>	Punto lógico de agarre para Movelt 2.
<code>`table_link -> tag_mesa`</code>	Ancla fija para la localización AprilTag.

Dato importante: El frame ``world`` existe como marco auxiliar porque el URDF importado del Kinova espera un padre con ese nombre.

■ ■ *Ejercicio: Visualiza el árbol*

1. Lanza: ``ros2 launch burger_description display.launch.py use_static_carts:=true``
2. Genera el diagrama TF: ``ros2 run tf2_tools view_frames``
3. Abre el ``frames.pdf`` resultante y ubica la cadena: ``map -> table_link -> world -> gen3_base_link``.
4. Ubica también la conexión temporal: ``tag_mesa -> tag_carrito1 -> car1_base_link``.

■ *Criterios de Éxito*

- Explicas por qué ``world`` no cuelga directamente de ``map``.
- Identificas que ``table_to_tag_mesa`` es la raíz de todo el sistema de localización de carritos.

■ ■ Mini-Reto de Validación: Rastreado el Origen

1. En una terminal, ejecuta este comando mágico para ver la relación entre dos piezas:

```
ros2 run tf2_ros tf2_echo map delivery_slot_1
```

2. **Analiza:** El resultado te dará una traslación (At) y rotación (Rotation).

3. **Compara:** Abre el URDF y busca el joint `staging\_to\_delivery\_slot\_1`. ¿Los valores de `xyz` coinciden con lo que dice la terminal?

4. *Pista: Si no coinciden exactamente, es porque TF2 suma todas las distancias desde el 'map' hasta el hijo. ¡Intenta hacer la suma tú mismo!*

5. Ajuste de Precisión: `origin`, `xyz` y `rpy`

■ El Concepto

Cada `joint` tiene un `origin` que define la transformación padre-hijo:

- `xyz`: desplazamiento en metros.
- `rpy`: rotación (Roll, Pitch, Yaw) en radianes.

> [!TIP]

> **Fundamento de Robótica: La Regla de la Mano Derecha**

> En ROS (y en la mayoría de la robótica), usamos sistemas de coordenadas "derechos".

> - El dedo **Pulgar** apunta al eje **X** (Rojo - hacia adelante).

> - El dedo **Índice** apunta al eje **Y** (Verde - hacia la izquierda).

> - El dedo **Medio** apunta al eje **Z** (Azul - hacia arriba).

> ¡Memoriza esto! Te ahorrará horas de frustración cuando intentes orientar un sensor o una cámara.

■ Revisión: El ancla `tag\_mesa`

Busca la conexión física del tag en la mesa:

```
rg -n 'table_to_tag_mesa' burger_description/urdf/delivery_scene_fixed.urdf
```

El patrón típico es:

```
<joint name="table_to_tag_mesa" type="fixed">
<origin xyz="0.5 0.3 0.025" rpy="0 0 0" />
</joint>
```

Interpretación: 50cm en X, 30cm en Y respecto al centro de la mesa, y 2.5cm elevado sobre la superficie.

■ ■ Tu Reto: Calibración Real

1. Identifica el origen de `table\_link` en RViz.

2. Mide físicamente la posición del centro del AprilTag real respecto al origen de la mesa.

3. Convierte a metros y actualiza `table\_to\_tag\_mesa` en el URDF.

4. Si el sticker está rotado, ajusta el valor de `yaw` (el último de los tres ceros en `rpy`).

■ Criterios de Éxito

- El frame `tag\_mesa` en RViz coincide exactamente con la posición del tag físico.
- Entiendes por qué no debes mover `tag\_carrito` para compensar un error que está en `tag\_mesa`.

■ ■ Mini-Reto de Validación: El Desafío del Offset

1. Busca el joint ``map_to_table`` en el URDF de la escena.
2. Predicción: Si cambiamos el valor de ``xyz="0.80 -1.00 0.80"`` a ``xyz="0.90 -1.00 0.80"``, ¿hacia dónde se moverá la mesa en RViz? (¿Adelante, atrás, izquierda o derecha?).
3. Prueba de fuego: Haz el cambio, compila (``colcon build``), relanza RViz y verifica si tu predicción fue correcta. ¡No olvides dejarlo como estaba después!

6. Carritos como URDFs Separados

■ El Concepto

Los carritos son móviles, pero su geometría interna es rígida. Separarlos de la escena fija resuelve dos responsabilidades:

1. El URDF del carrito define dimensiones, ruedas y offsets internos.
2. El Nodo de Localización define dónde está ese carrito en la mesa en cada instante.

■ Revisión en el Proyecto

Verifica que la escena fija no contenga los carritos:

```
rg 'car1_base_link' burger_description/urdf/delivery_scene_fixed.urdf
```

(Debería estar vacío).

Ahora inspecciona un carrito: ``code burger_description/urdf/car1_apriltag.urdf``.

Busca los joints que miden los offsets críticos:

- ``tag_carrito1_to_car1_base`` (Offset desde el sticker al centro del carro).
- ``car1_base_to_delivery_tray`` (Punto donde se entrega la comida).

■ ■ Ejercicio: Visualización con Carritos

1. Lanza: ``ros2 launch burger_description display.launch.py use_static_carts:=true``
2. En RViz, busca los displays de tipo ``RobotModel``. Deberías tener tres:
 - ``RobotModel`` (Escena/Kinova, topic ``/robot_description``).
 - ``Car1Model`` (Carrito 1, topic ``/car1/robot_description``).
 - ``Car2Model`` (Carrito 2, topic ``/car2/robot_description``).
3. Depuración: Si ves el frame ``tag_carrito1`` pero no el carro, verifica que el ``Description Topic`` en RViz sea el correcto.

■ Criterios de Éxito

- Visualizas los carritos temporalmente usando ``use_static_carts:=true``.
- Identificas qué joint ajustar si el AprilTag físico de un carrito no está perfectamente centrado.

■ ■ Mini-Reto de Validación: El Test de Independencia

1. Abre el launch ``display.launch.py`` y busca el parámetro ``use_static_carts``.
2. Experimento: Cambia el valor por defecto a ``False`` en el código (o lánzalo sin el parámetro en la terminal).
3. Observa: En RViz, ¿puedes ver la mesa y el robot? ¿Puedes ver los carritos?

4. **Pregunta:** Si los carritos no aparecen, ¿significa que el URDF del carrito está roto o que simplemente no hay nadie diciendo *dónde* está el carrito?

5. **Reflexión:** ¿Por qué esto es mejor que tener el carrito "soldado" a la mesa en el URDF fijo?

7. ¡Asegura tu trabajo! Compilación e Instalación

■ El Concepto

ROS 2 necesita que tus archivos estén en el espacio de instalación (`install`). Si creas un archivo nuevo o renombas uno y no reconstruyes el paquete, el sistema no lo encontrará.

■ Revisión: Xacro

Aunque usamos URDFs planos, ROS 2 permite usar Xacro para variables. Verifica que el sistema puede procesar tus archivos:

```
xacro burger_description/urdf/delivery_scene_fixed.urdf > /tmp/test.urdf
```

■ Ejercicio: El Ciclo de Desarrollo

Cada vez que hagas un cambio importante:

1. **Compila:** `cd ~/ros2_ws && colcon build --packages-select burger_description --symlink-install``
2. **Refresca:** ``source install/setup.bash``
3. **Verifica:** ``ls -l install/burger_description/share/burger_description/urdf``

■ Criterios de Éxito

- Entiendes que `src` es tu espacio de escritura y `install/share` es el espacio de lectura de ROS 2.
- El launch no falla con `FileNotFoundException` tras agregar un archivo nuevo.

8. ¡Verifica antes de lanzar! Herramientas de Validación

■ El Concepto

No confíes solo en RViz. Un URDF puede verse bien pero tener errores estructurales que romperán MoveIt 2 o el controlador.

■ Tu Kit de Herramientas Obligatorio

Ejecuta estas pruebas para cada archivo:

1. **Validación de XML:**

```
xmllint --noout burger_description/urdf/delivery_scene_fixed.urdf
```

2. **Validación de Árbol Cinemático:**

```
check_urdf burger_description/urdf/delivery_scene_fixed.urdf
```

(La raíz debe ser `map` para la escena y `tag\_carrito` para los carritos).

3. **Sintaxis del Launch:**

```
python3 -m py_compile burger_description/launch/display.launch.py
```

■ Criterios de Éxito

- Todas las herramientas reportan éxito (sin errores de sintaxis o de árbol).

9. MoveIt 2, Colisiones y `tool frame`

■ El Concepto

Movelt 2 necesita tres cosas para planificar movimientos seguros:

1. Cadena cinemática correcta.
2. Geometría de colisión útil.
3. Un frame de herramienta (`tool frame`) bien definido.

■ Revisión: El punto operativo

Busca el frame semántico de agarre en `delivery\_scene\_fixed.urdf`:

```
rg -n 'gen3_end_effector_link|burger_grip_frame|tool_to_grip'  
burger_description/urdf/delivery_scene_fixed.urdf
```

Observación: `burger\_grip\_frame` cuelga de `kinova\_tool\_frame` con un offset en Z. Este es el punto que Movelt 2 usará como objetivo para las hamburguesas.

■ Ejercicio de Reflexión

Si la hamburguesa real queda 5cm más alta que el punto asumido:

- **Incorrecto:** Deformar la malla visual o mover el `map`.
- **Correcto:** Ajustar el offset de `burger\_grip\_frame` en el joint `tool\_to\_grip`.

■ Criterios de Éxito

- Explicas la diferencia entre el link físico (`end\_effector\_link`) y el frame lógico de trabajo (`burger\_grip\_frame`).
- Entiendes por qué un objeto sin `` es invisible e intangible para Movelt 2.

10. Integración con AprilTags

■ El Concepto

El sistema de visión estima dónde están los carritos respecto a la cámara, pero Movelt 2 necesita saber dónde están respecto a la mesa. La fórmula lógica es:

```
T(tag_mesa -> tag_carrito) = inverse(T(camera -> tag_mesa)) * T(camera -> tag_carrito)
```

Después, el nodo de localización publica dinámicamente: `tag\_mesa -> tag\_carrito`.

■ Revisión en Guía de Visión

Abre `vision\_setup/LOCALIZACION\_APRILTAG.md` y verifica que la cadena cinemática coincide:

`tag\_mesa -> tag\_carrito1 -> car1\_base\_link -> car1\_delivery\_tray\_frame`

■ Ejercicio Final de Localización

1. **Lanza sin TFs temporales:** `ros2 launch burger\_description display.launch.py`
2. **Observa:** Los carritos no aparecen. Esto confirma que el sistema está listo para recibir datos reales del nodo de visión.
3. **Verifica:** Ejecuta `ros2 run tf2\_tools view\_frames`. El árbol estará desconectado en el tramo `tag\_mesa -> tag\_carrito` hasta que inicies el nodo de localización.

■ Criterios de Éxito

- Explicas por qué `tag\_carrito1` no debe ser un `joint fixed` dentro de la escena.
- Entiendes que si los carritos no aparecen en modo real, el primer lugar a revisar es la cámara y el nodo de detección, no el URDF.

■ ■ **Mini-Reto de Validación: El Detective de Transformadas**

1. Abre el archivo ``vision_setup/LOCALIZACION_APRILTAG.md``.
2. Busca la fórmula matemática de la transformada.
3. **Ejercicio Mental:** Si la cámara ve que el ``tag_mesa`` está a 1 metro de ella (en Z) y el ``tag_carrito1`` está a 1.2 metros (en Z), ¿a qué distancia está el carrito de la mesa aproximadamente?
4. **En el URDF:** Mira el joint ``table_to_tag_mesa``. Si movemos físicamente el AprilTag de la mesa pero no actualizamos este joint, ¿qué pasará con la ubicación de los carritos en RViz?

11. Lista de Chequeo Final (Checklist)

Antes de dar por válida tu configuración, asegúrate de haber hecho este recorrido:

1. ☐ Ejecutar ``xmllint`` y ``check_urdf`` sin errores.
2. ☐ Reconstruir el paquete y verificar archivos en ``install/share``.
3. ☐ Verificar en RViz que ``map`` es el Fixed Frame.
4. ☐ Confirmar que el brazo Kinova se mueve en el rango esperado (Joint State Publisher).
5. ☐ Verificar que la pinza Robotiq abre y cierra con ``mimic``.
6. ☐ Generar ``view_frames`` y validar la jerarquía completa.

■ ■ **¡No caigas en la trampa! (Errores Comunes)**

- **Olvidar reconstruir:** Cambiar el URDF en ``src`` y esperar que se vea en RViz sin hacer ``colcon build``.
- **Meshes no encontrados:** No haber creado el enlace simbólico ``meshes -> visual/meshes`` en la carpeta ``src``. Sin esto, la mesa y objetos fijos saldrán en rojo o serán invisibles.
- **Doble publicación:** Usar ``use_static_carts:=true`` al mismo tiempo que el nodo real de AprilTags.
- **Offsets mal ubicados:** Corregir un error de la mesa moviendo el carrito. Siempre calibra de la raíz hacia las hojas.
- **Colisiones complejas:** Usar mallas CAD pesadas para detección de choques en lugar de cajas simples.

■ **Resultado Esperado (Lo que ahora sabes)**

Al terminar esta guía, eres capaz de:

- Explicar cómo un URDF se convierte en el parámetro ``robot_description``.
- Validar la integridad de un modelo robótico con herramientas de terminal.
- Entender por qué separamos la escena de los objetos móviles.
- Ajustar offsets físicos (``xyz``, ``rpy``) para que el mundo virtual coincida con el real.
- Preparar el entorno para la planificación de movimientos con MoveIt 2.

¡Excelente trabajo! Estás listo para el siguiente paso: **¡Control e Inteligencia con MoveIt 2!**